

NEXUS

Autonomous Research, Decision & Execution Agent — Project Explainer

"From Idea to Execution — an AI Agent That Thinks, Plans, Acts, Learns, and Adapts."

This document explains NEXUS using the standard 10-point project framework (Problem, Objective, User, Input, Process, Output, Tech Stack, Architecture, Limitations, Future Scope), followed by a detailed breakdown of exactly what has been built so far — every tab, every backend module, and what each one does.

1 Problem Statement

Problem kya hai?

Planning a real project (e.g. an AI/ML build) manually is hard once real-world constraints enter the picture — limited days, limited hours per day, a fixed budget, no GPU, a small team. Most planning tools produce a single static roadmap and stop there. They don't justify *why* a technology was chosen, don't adapt when a constraint changes mid-project (GPU turns out unavailable, budget gets cut, a teammate leaves), and don't let you preview the impact of a change before committing to it.

2 Objective

Project ka goal kya hai?

Build an agent that goes beyond planning: it researches trade-offs with evidence, has specialist agents **debate** each key technical decision against the user's real constraints, decomposes the goal into a scaled, assignable task plan, lets the user **simulate what-if scenarios** before committing, and **self-corrects** the plan when a real blocker is reported — closing the loop from Goal to Execution instead of stopping at a document.

3 User

Kiske liye banaya hai?

- Students and hackathon participants planning a time-boxed AI/ML or software project.
- Solo builders / indie developers with a fixed budget and no dedicated team.
- Small project teams who need tasks split by skill and a plan that survives real-world surprises.

4 Input

System ko kya data milega?

- **Goal** — free text, e.g. "Build an AI Healthcare diagnosis assistant".
- **Days available** and **hours per day** — defines the real time budget.
- **Budget (■)** — drives every cost-sensitive decision.

- **GPU availability** (yes/no) — drives the AI-hosting decision.
- **Team** (optional) — name + free-text skills per member, used for task assignment.
- Later: a **blocker report** (free text) describing what actually happened, e.g. "my teammate left and budget is now zero".

5 Process / Logic

Input ke baad system kya steps follow karega?

GOAL → UNDERSTAND CONSTRAINTS → RESEARCH (evidence gathering) → MULTI-AGENT DEBATE (Research / Cost / Technical / Risk agents argue, Final Decision agent synthesizes) → DECOMPOSE into phases & tasks (auto MVP-mode if time is tight) → ASSIGN tasks to team by skill → PLAN delivered → [optional] WHAT-IF SIMULATION → [optional] BLOCKER REPORTED → DETECT → RE-DEBATE → DIFF → REPLAN.

6 Output

System user ko kya provide karega?

- A phase-by-phase, hour-estimated **task plan** with assignees and status tracking.
- **Evidence-based research cards** — recommendation, reasoning, supporting evidence, alternative considered.
- A full **multi-agent debate transcript** per decision, with a final decision and rationale.
- **What-if comparisons** — what changes if the deadline, budget, GPU, or team size shifts.
- A **narrated replanning log** whenever a blocker is reported, showing exactly what changed and why.

7 Technology Stack

Layer	Technology	Notes
Frontend	React 18 + Vite	Custom CSS design system, no UI kit
Backend	Python + FastAPI + Pydantic	REST API, in-memory mission store
Reasoning engine	Rule-based today; Claude-ready	Pluggable provider — swap via env var, no code change elsewhere
Dev tooling	Uvicorn, npm/Vite dev server	Runs locally on ports 8420 (API) / 5174 (UI)

8 Architecture

Components kaise connected hain?

```
User
|
v
React Frontend (Vite, localhost:5174)
|  fetch /api/mission/*
v
```

```
FastAPI Backend (localhost:8420)
|
v
Orchestrator (run_pipeline)
|-- Research Agent      (evidence-based recommendations)
|-- Multi-Agent Debate  (Research/Cost/Technical/Risk -> Final Decision)
|-- Decomposition Agent (phases/tasks, MVP-mode scaling, team assignment)
|-- Simulation Agent    (what-if, non-persisted)
|-- Replanning Agent    (blocker -> constraint diff -> re-run -> decision diff)
|
v
In-memory Mission Store --> JSON response --> Frontend renders 6 tabs
```

9 Limitations

Project kya nahi kar sakta (abhi)?

- Reasoning is currently **rule-based / deterministic**, not a real LLM call — recommendations come from a fixed knowledge base of 5 decision topics, not open-ended generation for any arbitrary domain.
- **No database** — missions live in an in-memory dict; a server restart clears all mission history.
- **No authentication / multi-user isolation** — anyone hitting the API can read/modify any mission by ID.
- Blocker parsing uses **keyword and regex heuristics**, not true language understanding — unusual phrasing of a blocker may go undetected.
- **Team dependency/delay cascade detection** (e.g. "frontend delay affects integration") from the original concept is not implemented — only skill-based task assignment exists today.
- AI-generated recommendations should still be **human-verified** before being treated as final decisions.

10 Future Scope

Aage kya improve kar sakte hain?

- Swap the mock reasoning engine for real Claude API calls (already wired via `llm_provider.py` — just set `ANTHROPIC_API_KEY + LLM_PROVIDER=claude`).
- Persistent database (PostgreSQL) for mission history, plus basic auth for multi-user use.
- Team dependency/delay-cascade detection and automatic task reallocation.
- Export the plan to PDF / Notion / Trello; calendar integration for deadlines.
- Voice interaction and real-time progress notifications.
- User-defined custom what-if scenarios instead of only the 5 preset ones.

What We've Built So Far

A working, browser-verified prototype — component by component.

The flow implemented end-to-end: **Goal** → **Understand** → **Plan** → **Break into Tasks** → **Execute** → **Monitor** → **Adapt** → **Complete**. Everything below has been run and visually verified in the browser, not just written.

Frontend — Tab by Tab

Tab	Component file	What it does
Intake form	IntakeForm.jsx	Collects goal, days, hours/day, budget, GPU checkbox, team members (name + skills). Example-goal chips for quick demo.
■■ Plan	PlanPanel.jsx	Shows the phase-by-phase task breakdown with hours, assignee, and a status dropdown (pending/in progress/done/blocked). Per-phase progress bar updates live as tasks are marked done.
■ Debate	DebatePanel.jsx	For each of 5 architecture decisions, shows all 4 agent opinions (Research/Cost/Technical/Risk) side by side, plus a highlighted Final Decision card with rationale.
■ Research	ResearchPanel.jsx	Evidence-based recommendation cards: recommendation, reasoning, bullet evidence (derived from the debate), and the alternative that was considered.
■ What-if	ScenarioPanel.jsx	5 one-click scenario buttons (7-day deadline, no GPU, ■0 budget, solo dev, double budget). Runs the pipeline against a hypothetical change WITHOUT touching the live plan, then shows a delta summary.
■■ Adapt	BlockerPanel.jsx	Free-text "report a blocker" box. Triggers the self-correcting agent: detects the affected constraint, re-runs the full pipeline, and narrates exactly what changed and why — the plan updates live.
■ Log	LogPanel.jsx	Full mission narrative — from initial goal receipt through every replanning event.

Backend — Module by Module

File	Responsibility
knowledge_base.py	Constraints model — goal, days, hours/day, budget, GPU, team; derives total_hours, is_tight_timeline, is_zero_budget, etc.
agents/debate.py	5 decision topics (AI hosting, database, backend framework, deployment, frontend). Each generates 4 agent opinions and a constraint-aware final decision with rationale.
agents/research.py	Turns each debate decision into an evidence-based recommendation card.
agents/decomposition.py	Builds phases & tasks scaled to available hours; auto-switches to a 4-phase MVP mode under a tight timeline; assigns tasks to team members by skill match.

agents/simulate.py	Re-runs the pipeline against a hypothetical constraint change (5 preset scenarios) without persisting, and diffs it against the live plan.
agents/replan.py	The self-correcting loop: parses free-text blocker reports into concrete constraint changes (GPU, budget, timeline, team), re-runs the pipeline, and diffs decisions before/after.
orchestrator.py	Ties Research → Debate → Decomposition into one MissionResult; the single "spine" of the goal-to-execution pipeline.
llm_provider.py	Pluggable reasoning layer. Ships with a deterministic MockProvider (offline, no API key). A ClaudeProvider stub is ready — set env vars to switch, no other code changes.
routers/mission.py	REST endpoints: create mission, get mission, list/run what-if scenarios, report a blocker, update task status.

UI Theme

Two full theme passes were applied on top of the working prototype: **Frosted Blue** (Ice Blue background, Royal Blue primary, Aqua highlight) and then the current **Peach Sunset** (Peach Cream background #FFF7ED, white frosted-glass cards, Coral Orange primary #F97316, Peach secondary #FB923C, Rose Pink highlight #EC4899, Brown text #431407). A dark variant exists in code but the app is now locked to **light mode by default** regardless of the OS color-scheme preference, per your last instruction.

Verification Done

- Backend logic smoke-tested directly (pipeline, scenarios, blocker parsing) before wiring the API.
- FastAPI endpoints tested via curl: create mission, list scenarios, simulate, report blocker.
- Full browser walkthrough: submitted a goal, checked all 6 tabs render correctly, updated a task's status and watched the progress bar move, ran a what-if scenario and saw decisions flip, reported a real blocker ("teammate left + budget to zero") and watched the self-correcting agent re-debate and update the plan live.
- No console errors in any of the tested flows, in both the Frosted Blue and Peach Sunset themes.

Code location: C:\Users\rajes\nexus-agent — backend on port 8420, frontend on port 5174. See README.md in that folder for run instructions and the full demo script.